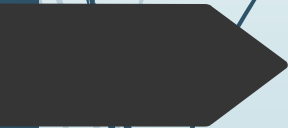


Desarrollo web en entorno servidor

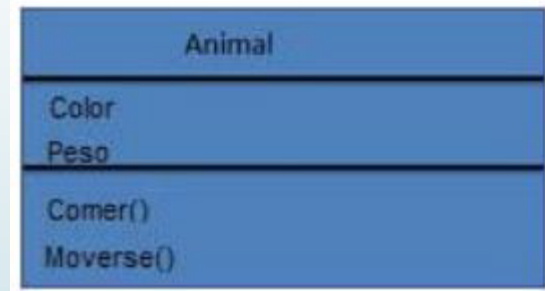


Progración orientada a
Objetos



LA CLASE

- Es para fabricar objetos partiendo de un modelo.
- Los objetos tienen sus propios:
 - Atributos
 - Métodos



```
<?php
class Animal // palabra clave class seguida del nombre de la clase.
{
    // Declaración de atributos y métodos.
}
?>
```

LA CLASE

Hay tres tipos de palabras clave para definir la visibilidad de un atributo o de un método:

- **Private:** Solo el código de su clase puede ver y acceder al atributo o método.

```
<?php
    class Animal // palabra clave seguida del nombre de la clase.
    {
        // Declaración de atributos.
        private $color = "gris";
        private $peso = 10;
    }
?>
```

LA CLASE

- **Public:** Todas las demás clases pueden acceder a este atributo o método.

```
public function comer()  
{  
    //Método posible para acceder a las propiedades  
    //color y peso  
}  
  
public function moverse()  
{  
    //Método posible para acceder a las propiedades  
    //color y peso  
}
```

LA CLASE

- **Protected:** Solo el código de su clase y de sus subclases pueden acceder a este atributo o métodos.

AÑADIR UN MÉTODO

- Añadir código a la clase significa aplicar la clase.
- Para acceder a los atributos se debe utilizar la pseudovariable **\$this** (representa el objeto).
- Para acceder al atributo o método del objeto se utiliza el operador **->**

```
<?php
class Animal // palabra clave seguida del nombre de la clase.
{
    // Declaración de atributos y métodos.
    private $color = "gris";
    private $peso = 10;
```

```
public function añadir_un_kilo()
{
    $this->peso = $this->peso + 1;
}
```

UTILIZACIÓN

- Para utilizar una clase que está en otro fichero se debe incluir en el fichero que se quiera utilizar.
- Para ello se utiliza **include**

```
include('Animal.class.php');
```

```
//instanciar la clase Animal  
$perro = new Animal();
```

ACTUALIZACIÓN Y LECTURA DE ATRIBUTOS

- El principio de encapsulación requiere que en general los atributos sean privados.
- Se necesitan métodos públicos para leer y escribir dichos atributos.
- Generalmente dichos métodos se denominan **set** (para escribir) y **get** (para leer)

```
// Declaración de atributos  
private $color = "gris";  
private $peso = 10;
```

```
//accesos  
public function getColor()  
{  
    return $this->color; //devuelve el color  
}  
public function setColor($color)  
{  
    $this->color = $color; //escrito en el atributo color  
}
```

PASO DE ARGUMENTO TIPO OBJETO

- Los métodos pueden tomar diferentes tipos de argumentos, entre ellos objetos.

```
// Declaración de atributos
private $color = "gris";
private $peso = 10;
```

```
//Métodos
public function comer_animal(Animal $animal_comido)
{
    //el animal que come aumenta su peso tanto
    //como el del animal comido
    $this->peso = $this->peso + $animal_comido->peso;
    //el peso del animal comido y su color se restablecen a 0
    $animal_comido->peso = 0;
    $animal_comido->color = "";
}
```

PASO DE ARGUMENTO TIPO OBJETO

- Llamada al método:

```
//instanciar la clase Animal  
$gato = new Animal();
```

```
//instanciar la clase Animal  
$pez = new Animal();
```

```
//el gato come al pez  
$gato->comer_animal($pez);
```

- Maneras inadecuadas de llamada:

```
$gato->comer_animal("Rana");
```

```
$gato->comer_animal(4);
```

EL CONSTRUCTOR **__construct()**

- Sirve para construir un objeto de tipo clase.
- Cuando se escribe `new NombreDeClase` por defecto se llama al constructor.
- Se designa **__construct** y no tiene **return**

```
public function __construct ($color, $peso)
//Constructor que solicita 2 argumentos.
{
    echo 'Llamar al constructor.<br />';
    $this->color = $color; // Inicialización del
                        // color.
    $this->peso = $peso; // Inicialización del peso.
}

//instanciar la clase Animal con su constructor
$perro = new Animal("beige",7);
```

EL CONSTRUCTOR `unset()`

- Sirve para destruir el objeto para liberar la memoria.
- Se llama automáticamente al final del script o cuando se destruye el objeto.
- Para destruir el objeto se utiliza **`unset()`**, toma como argumento un objeto.

```
public function __destruct()  
{  
    echo 'Llamar al destructor';  
}
```

```
//destrucción del objeto  
unset($perro);
```

LAS CONSTANTES DE CLASE

- Es similar a una constante normal.
- Un valor asignado a un nombre que no cambia.
- Para llamar a las constantes de clase se escribe `::` entre la clase y la constante.

```
<?php
class Animal
{
    // Declaración de atributos
    private $color = "gris";
    private $peso = 10;

    //constantes de clase
    const PESO_LIGERO = 5;
    const PESO_MEDIO = 10;
    const PESO_PESADO = 15;
```

```
public function __construct ($color, $peso)
```

```
//instanciar la clase Animal con su constructor
$pez1 = new Animal("gris",Animal::PESO_MEDIO);
$pez2 = new Animal("rojo",Animal::PESO_LIGERO);
```

MÉTODOS ESTÁTICOS

- Está unido a la clase, pero no al objeto.
- Se utiliza la palabra **static** delante de la **function**.
- En dicho método no se puede utilizar la palabra clave **\$this** ya que representa el objeto.
- Para su llamada se utiliza **::** como con las constantes.
- Se puede llamar al método desde el objeto, pero no cambia nada de dicho objeto.

```
public static function moverse()  
{  
    echo "El animal se mueve.";  
}
```

```
//llamada al método estático  
Animal::moverse()
```

ATRIBUTOS ESTÁTICOS

- Es un atributo propio de la clase y no del objeto.
- Es una variable y se puede cambiar su valor desde los objetos.
- Se escribe añadiendo la palabra **static** delante del nombre.
- Por ejemplo, para representar un contador de cuantas veces se instancia una clase, se pondría en el constructor.
- No se puede utilizar **\$this** para cambiar el valor porque está unido a la clase.
Se debe utilizar la palabra clave **self**.

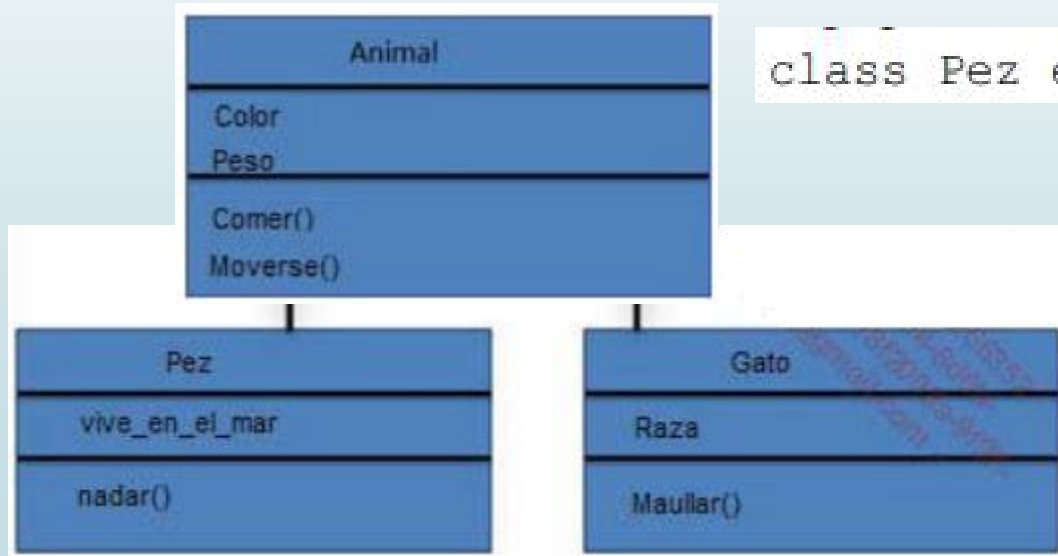
ATRIBUTOS ESTÁTICOS

```
// Declaración de la variable estática $contador  
private static $contador = 0;
```

```
self::$contador = self::$contador + 1;
```

LA HERENCIA

- Permite reutilizar el código de una clase.
- Clase hija hereda atributos y métodos no privados de la clase madre.
- Para ello se utiliza la palabra clave **extends**



```
class Pez extends Animal
```

LA HERENCIA Protected

- Equivale a private salvo para las clases hijas que pueden ver los atributos protected de la clase madre.

```
// Declaración de atributos  
private $color = "gris";  
private $peso = 10;  
protected $edad = 0;
```

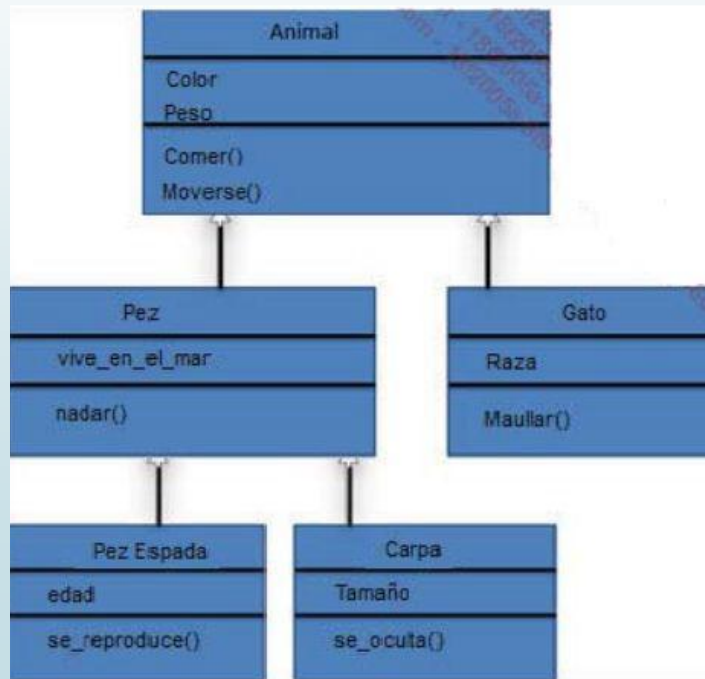
LA HERENCIA Sustitución

- Es para modificar un método que ya existe en una clase madre.
- El método existe en dos clases diferentes con el mismo nombre que según el contexto se ejecuta el de la hija o el de la madre.
- **parent** es la palabra clave que designa a la clase madre.

```
padre::comer_animal($animal_comido);
```

HERENCIA EN CASCADA

- La herencia múltiple no existe. Una clase solo puede heredar de una clase y esa clase a su vez de otra clase.



CLASES ABSTRACTAS

- En PHP, una **clase abstracta** es una clase que **no puede ser instanciada directamente**.

Se utiliza como **plantilla base** para que otras clases hereden de ella.

Sirve para **definir atributos y métodos comunes** y, además, para obligar a las subclases a implementar ciertos métodos.

Declaración:

```
abstract class Figura { ... }
```

CLASES ABSTRACTAS

- En PHP, una **clase abstracta** es una clase que **no puede ser instanciada directamente**.

Se utiliza como **plantilla base** para que otras clases hereden de ella.

Sirve para **definir atributos y métodos comunes** y, además, para obligar a las subclases a implementar ciertos métodos.

Declaración:

```
abstract class Figura { ... }
```

CLASES ABSTRACTAS : Ejemplo

- Figura.php

```
<?php
// Clase abstracta base
abstract class Figura {
    protected $color;

    public function __construct($color) {
        $this->color = $color;
    }

    // Método abstracto (obliga a las subclases a implementarlo)
    abstract public function calcularArea();

    // Método normal (se hereda tal cual)
    public function getColor() {
        return $this->color;
    }
}
```

CLASES ABSTRACTAS : Ejemplo

- Circulo.php

```
<?php
require_once 'Figura.php';

class Circulo extends Figura {
    private $radio;

    public function __construct($color, $radio) {
        parent::__construct($color);
        $this->radio = $radio;
    }

    public function calcularArea() {
        return pi() * pow($this->radio, 2);
    }
}
```

CLASES ABSTRACTAS : Ejemplo

- Rectangulo.php

```
<?php
require_once 'Figura.php';

class Rectangulo extends Figura {
    private $ancho;
    private $alto;

    public function __construct($color, $ancho, $alto) {
        parent::__construct($color);
        $this->ancho = $ancho;
        $this->alto = $alto;
    }

    public function calcularArea() {
        return $this->ancho * $this->alto;
    }
}
```

CLASES ABSTRACTAS : Ejemplo

- Principal.php

```
<?php
require_once 'Circulo.php';
require_once 'Rectangulo.php';

// Crear objetos
$circulo = new Circulo("rojo", 5);
$rectangulo = new Rectangulo("azul", 4, 6);

// Mostrar resultados
echo "El círculo de color " . $circulo->getColor() . " tiene área: " . $circulo->calcularArea() . "\n";
echo "El rectángulo de color " . $rectangulo->getColor() . " tiene área: " . $rectangulo->calcularArea() . "\n";

// Ejemplo de polimorfismo
echo "\n--- Usando polimorfismo ---\n";
$figuras = [$circulo, $rectangulo];

foreach ($figuras as $figura) {
    echo "Figura de color " . $figura->getColor() . " → Área: " . $figura->calcularArea() . "\n";
}
```